

Writing an AI-Native Dissertation

Session format: Training session — 4-hour (half-day), hands-on

Presenter: Damian Betebenner, Center for Assessment

Theme alignment: Measurement Science in AI-Integrated Assessment and Pedagogy

Program Abstract

44 / 50 words

A hands-on, half-day session in which participants build their own AI-native dissertation: a reproducible project that produces a website, a thesis PDF, an interactive application, and an API queryable by AI agents. Participants leave with a personalized environment generated from pre-session research scoping responses.

Session Summary

800 / 800 words

Learning Objectives

By the end of the session, participants will be able to:

- Explain the dissertation as a multi-modal knowledge-transmission platform, and articulate how an AI-native workflow changes its reach, reproducibility, and the locus of human responsibility.
- Run and navigate their own dissertation environment — an R package, Quarto sources, and a NextJS application — generated in advance from their pre-session scoping responses.
- Render a single source to three outputs: an interactive website, a formatted thesis PDF, and a working paper.
- Apply the write-render-review cycle to draft and revise dissertation content with AI as a collaborator, while retaining full editorial control.
- Link narrative prose to live analysis so that reported values regenerate from code rather than being transcribed by hand.
- Build and publish their dissertation to the open web: the Quarto site as a public website and the interactive application deployed to Vercel.
- Expose their research through a REST API and Model Context Protocol (MCP) tools so that AI agents can query their analyses and data directly.

- Develop practical fluency in advanced AI tool use — a skill distinct from understanding how the models work internally — including customizing the AI harness (project rules, context files, and MCP tools) to a particular scholarly purpose.
- Document AI use transparently and apply emerging norms for attribution, reproducibility, and institutional policy.

Content and Relevance

The dissertation is, at root, a technology for transmitting understanding from one mind to others — and for six centuries its fundamental mode has barely changed. Knowledge passed from the oral disputation of the medieval university, to hand-copied manuscripts, to the printing press, to the PDF; each step widened reach but preserved the same modality: one human writes, and one human reads, in linear text. The digital era digitized the container, not the knowledge — a PDF is a photocopy of a printed page, with data and code still hidden “available upon request.” Measured against how scholarship is now produced and consumed, that mode is increasingly outdated. Two developments change the equation: APIs make findings addressable rather than merely readable, and AI agents mean the consumers of research are no longer exclusively human.

This session teaches a concrete, open, forkable framework that treats the dissertation as a platform: content and analysis are authored once and, driven by a single declarative specification and shared content layer, surfaced as a website, a thesis PDF, a working paper, a REST API, MCP tools, and an interactive application. The architecture is deliberately ordinary underneath (a standard R package, Quarto, a NextJS app, Git), so participants learn transferable practices rather than a bespoke tool. The environment is ready-to-hand — the tooling recedes so the author dwells in the dissertation, not the interface. The recurring discipline is “leave the technical plumbing to the AI, and own the content”: research questions, analysis, and interpretation remain the scholar’s responsibility. The context an author invests up front — framing, advisor writings, key literature, style exemplars — becomes an AI-queryable knowledge layer, because AI’s usefulness compounds with the depth of context it can draw on.

Fluency with AI is fast becoming an essential professional skill for measurement professionals, and there is no better place to build it than an extended, high-stakes scholarly project such as a dissertation or thesis, where genuine judgment — not toy exercises — is on the line. A central aim of the session is therefore practical, advanced tool use — directing AI well and customizing its harness of project rules, context files, and MCP tools to scholarly ends — a competence distinct from understanding how the models work internally.

The session operationalizes the measurement-science values at the center of the conference theme. Reproducibility underwrites the validity of every reported number, because results regenerate from versioned code with fixed seeds. Transparency follows from open, auditable analysis functions and logged AI interactions. Whatever the system derives or generates on the author’s behalf — the project specification, the scaffolding, AI-suggested prose — is inspectable, contestable, and exportable, so human agency is preserved and AI remains a collaborator and an access layer, never an author. Attribution and disclosure are built into the workflow. The framework and its onboarding are open source; the session is about the practice of AI-native scholarship, not any product.

Presenter's Expertise

Damian Betebenner (Center for Assessment) is the sole presenter. He is a measurement methodologist and data scientist best known for developing the Student Growth Percentile (SGP) methodology now used in many U.S. state accountability systems, and for authoring and maintaining the widely used open-source SGP R package ecosystem. He designed the AI-native dissertation framework and its pre-session onboarding system described here, and works daily in R, Quarto, reproducible-research tooling, and AI-integrated development. He has presented and taught this material to measurement audiences, including the vo.1 of this session at the NCME 2026 Annual Meeting.

Detailed Schedule

313 / 500 words

Preferred format: 4-hour (half-day).

Before the session (about 2 hours, completed online)

Participants complete a structured scoping questionnaire — the session application — that captures their research question, methodology, chapter outline, committee, key literature (uploaded PDFs), institution, and thesis-formatting requirements. Their responses generate a personalized starter environment (R package, Quarto sources, and NextJS application) so each participant arrives with a working dissertation specific to their own research rather than shared sample data.

In session (240 minutes)

- **0:00–0:25 (25 min)** — Welcome, framing, and an “AI is like a...” mental-models exercise. The presenter frames the dissertation as a knowledge-transmission mechanism; attendees share and discuss.
- **0:25–0:55 (30 min)** — The AI-native framework: an architecture walkthrough (R package to REST API to MCP to NextJS; Quarto single source to three outputs). The presenter demonstrates while attendees locate each layer in their own generated repository.
- **0:55–1:40 (45 min)** — Hands-on 1: tour and run your environment; render the website, the thesis PDF, and the working paper. Attendees work; the presenter circulates.
- **1:40–1:55 (15 min)** — Break.
- **1:55–2:55 (60 min)** — Hands-on 2: content creation with AI. The write-render-review cycle on a real chapter, and linking narrative to live analysis. Short instructional bursts alternate with sustained guided practice.
- **2:55–3:25 (30 min)** — Hands-on 3: build the site and deploy the interactive application to the open web (Vercel). Guided practice from each participant’s pre-provisioned environment, with a known-good fallback checkpoint so no one is blocked.
- **3:25–3:45 (20 min)** — Your dissertation as a queryable platform: a live demonstration of exposing the REST API and MCP tools to AI agents against a pre-provisioned environment,

followed by a guided attendee try.

- **3:45–4:00 (15 min)** — Wrap-up: AI ethics, attribution, reproducibility, next steps, and open Q&A.

Balance. Roughly half the session is hands-on practice on each participant’s own dissertation, complemented by short instructional demonstrations and continuous question-taking, with a dedicated closing Q&A.

Prior Presentation and Improvements

An earlier version (vo.1) of this session was presented at the NCME 2026 Annual Meeting (April, 2026, Los Angeles, CA). This v1.0 substantially revises that session: (1) a roughly 2-hour pre-session scoping questionnaire now generates each participant’s personalized application in advance, so the session is spent on the participant’s own dissertation rather than shared sample data; (2) hands-on build-and-deploy to the open web has been added; (3) exposing the dissertation through a REST API and MCP tools to AI agents has been added; and (4) the treatment of reproducibility, attribution, and AI ethics has been strengthened.

Software Requirements

Software needed

A modern web browser; Git; R (≥ 4.5); Quarto (≥ 1.6); Node.js (LTS) with pnpm; a free GitHub account; and, optionally, a free Vercel account for deployment. RStudio or VS Code is recommended as an editor. Most importantly, an AI coding assistant (for example, Claude Code, Cursor, or Codex) — the tool participants point at their repository to make changes throughout the session. With the single exception of the AI assistant, all required software is free and open-source.

Installation / payment

Participants install the above ahead of time using setup instructions emailed before the session, and complete the pre-session scoping questionnaire. Every component except one is free: Git, R, Quarto, Node.js/pnpm, and the GitHub and optional Vercel accounts cost nothing. The one paid requirement is the AI coding assistant — the engine that reads and updates the dissertation repository. Sustained, agentic use during the session realistically requires either an active paid subscription (for example, Claude Pro/Max, ChatGPT Plus, or Cursor Pro — roughly \$20/month) or pay-as-you-go API credits from a provider such as OpenAI or Anthropic. Free tiers exist but are typically too rate-limited for hands-on work, so participants should arrive with an active paid plan or a small amount of API credit already configured.

Hardware

A laptop (macOS, Windows, or Linux) on which the participant can install software (adminis-

trator rights), with roughly 5 GB of free disk space and reliable Wi-Fi. No specialized hardware is required.